Automated build and Make practical

David Henty, Bartosz Dobrzelecki, Adrian Jackson and Mike Jackson

12 July 2016

1 Introduction

For an introductory tutorial to Make's concepts and syntax, please see the Software Carpentry lesson:

Mike Jackson (ed.): "Software Carpentry: Automation and Make." Version 2016.06, June 2016, https://github.com/swcarpentry/make-novice, 10.5281/zenodo.57473.

In the rest of this document, features of Make that are of use for C (and C++) and Fortran development are highlighted.

2 Built-in macros

Make has many predefined macros, or implicit variables, for C or Fortran developers e.g. from https://www.gnu.org/software/make/manual/html_node/Implicit-Variables.html:

- `CC`: Program for compiling C programs; default `cc`.
- `CXX`: Program for compiling C++ programs; default `g++`.
- `FC`: Program for compiling or pre-processing Fortran programs; default `f77`.
- `CFLAGS`: Extra flags to give to the C compiler.
- `CXXFLAGS`: Extra flags to give to the C++ compiler.
- `CPPFLAGS`: Extra flags to give to the C pre-processor and programs that use it (the C and Fortran compilers).
- `FFLAGS`: Extra flags to give to the Fortran compiler.
- `LDFLAGS`: Extra flags to give to compilers when they are supposed to invoke the linker, `d`, such as `-L`.
- `LDLIBS`: Library flags or names given to compilers when they are supposed to invoke the linker, `ld`.

As an example of their use:

```
scheduler: ${OBJ} ${LIB}

    ${CC} ${CPPFLAGS} ${CFLAGS} ${OBJ} ${LDFLAGS} -o $@
```

Be careful using these, they can vary across Make versions.

3 Pattern substitution macros

Special pattern substitution macros allow file names to be auto-generated e.g.

```
SRC = ${OBJ:.o=.c}
```

This sets the value of `SRC` to be everything in `OBJ` with `.o` replaced by `.c`. So, given:

```
OBJ = main.o schedule.o optimise.o io.o utils.o
```

`SRC` would be set to:

```
main.c schedule.c optimise.c io.c utils.c
```

4 Implicit rules

Implicit rules define general ways of rebuilding files of one type of file from another. A common example is a rule defining how to rebuilt object files from source files in C or Fortran.

4.1 Suffix rules

Suffix rules have a target of form:

```
.fromSuffix.toSuffix:
```

For example, the following rule defines how to create `.o` files from `.c` files using a C compiler:

```
.c.o:

    ${CC} ${CPPFLAGS} ${CFLAGS} -c $<
```

Note the action of this rule uses the automatic variable `$<`, which means "the first dependency of the current rule". However, the rule has no dependencies. The first dependency is implicitly assumed to be the `.c` file. So, for example, when running:

```
$ make code.o
```

The first dependency would implicitly be `code.c` and the action run would be:

```
${CC} ${CPPFLAGS} ${CFLAGS} -c code.c
```

For example, the following rule defines how to create `.o` files from `.f90` files using a Fortran compiler:

```
.f90.o:

    $(F90) -c $< $(LIB)
```

Many versions of Make, pre-define implicit rules for common file types. See, for example, https://www.gnu.org/software/make/manual/html_node/Catalogue-of-Rules.html.

4.2 Pattern rules

Pattern rules are more flexible than suffix rules. They have the general form:

```
T_PREFIX%T_SUFFIX: D_PREFIX%D_SUFFIX [OTHER DEPENDENCIES ...]

    BUILD RULE

    ...
```

For example, the following rule defines how to parse `xhtml` files in a `src` directory into `html` files:

```
%.html: src/%.xhtml ../dtd/main.dtd Makefile

    xparse $< > $*.html
```

Given a directory `src` with files `index.xhtml`, `chapter1.xhtml` and `chapter2.xhtml` the same rule can be used to create `index.html`, `chapter1.html` and `chapter2.html` in the current directory.

Pattern rules are an extension added in GNU Make to provide an easier-to-follow syntax.

5 A sample Makefile

Here is a sample Makefile for building a C program:

```
# Pre-processing options.

CFLAGS = -g

CPPFLAGS = -I./include -DDIAGNOSTICS

LDFLAGS = -L./lib -lrng -lm



# Object files.

OBJ = main.o schedule.o optimise.o io.o utils.o



# Include files.

INC = schedule.h io.h structs.h



# Source files - defined via a pattern substitution macro.

SRC = ${OBJ:.o=.c}
```

```
# A dependency chain.

${OBJ}: ${INC} Makefile



.PHONY: clean

clean:

    rm -f scheduler ${OBJ}



.c.o:

    ${CC} ${CPPFLAGS} ${CFLAGS} -c $<



scheduler: ${OBJ}

    ${CC} ${CPPFLAGS} ${CFLAGS} ${OBJ} ${LDFLAGS} -o $@
```

5.1 Extending the Makefile

Makefiles can start simple and grow as needed. In the above, we define:

```
LDFLAGS = -L./lib -lrng -lm
```

`m` is a standard C math library. `rng` is a library, for the purpose of this example, that we have written. This is assumed to be in `lib`. If we assume the source code for the library is in `lib/uni.c`, then we can extend the Makefile to build the library too.

We can add a macro defining the library file name:

```
LIB = lib/librng.a
```

We can then add a macro specifying the object file for our library's source code:

```
LOBJ = lib/uni.o
```

We can now add a rule to create our library from the object file, using the `ar` archive command:

```
${LIB}: ${LOBJ}

    ar -r $@ ${LOBJ}
```

Now we need a rule to compile `lib/uni.c` into `lib/uni.o`. We have our rule to compile C code already:

```
.c.o:

    ${CC} ${CPPFLAGS} ${CFLAGS} -c $<
```

But, by default, the C compiler creates its object files in the current directory, so if we were to run:

```
$ make lib/uni.o
```

it would create `uni.o` in the current directory, not in `lib` which is where we want it to be. We need to explicitly tell the C compiler where to create its output files which we can do by using the C compiler's `-o` flag to specify the output file. We can then use a special macro to create the desired output file name `lib/uni.o` from the C file name `lib/uni.c`:

```
${<:.c=.o}
```

This takes first, implicit, dependency of the current rule {denoted by `<`) and replaces its `.c` suffix with `.o`. So, running:

```
$ make lib/uni.o
```

would cause the following action to be run:

```
cc -DDIAGNOSTICS -g -o lib/uni.o -c lib/uni.c
```

Updating our suffix rule, with these changes, gives:

```
.c.o:

    ${CC} ${CPPFLAGS} ${CFLAGS} -o ${<:.c=.o} -c $<
```

Finally, we update the `scheduler` target, to add `LIB` as a dependency, so if our library changes, `scheduler` will be rebuilt:

```
scheduler: ${OBJ} ${LIB}

    ${CC} ${CPPFLAGS} ${CFLAGS} ${OBJ} ${LDFLAGS} -o $@
```

Our updated Makefile is:

```
# Pre-processing options.

CFLAGS = -g

CPPFLAGS = -I./include -DDIAGNOSTICS

LDFLAGS = -L./lib -lrng -lm



# Library files.

LIB = lib/librng.a



# Object files.
```

```
OBJ = main.o schedule.o optimise.o io.o utils.o

LOBJ = lib/uni.o


# Include files.

INC = schedule.h io.h structs.h


# Source files - defined via a pattern substitution macro.

SRC = ${OBJ:.o=.c}


# A dependency chain.

${OBJ}: ${INC} Makefile


.PHONY: clean

clean:

    rm -f scheduler ${OBJ}


.c.o:

    ${CC} ${CPPFLAGS} ${CFLAGS} -o ${<:.c=.o} -c $<


${LIB}: ${LOBJ}

    ar -r $@ ${LOBJ}


scheduler: ${OBJ} ${LIB}

    ${CC} ${CPPFLAGS} ${CFLAGS} ${OBJ} ${LDFLAGS} -o $@
```

If we now were to run this Makefile we'd get:

```
$ make scheduler

cc -DDIAGNOSTICS -g -o main.o -c main.c
```

```
cc -DDIAGNOSTICS -g -o schedule.o -c schedule.c

cc -DDIAGNOSTICS -g -o optimise.o -c optimise.c

cc -DDIAGNOSTICS -g -o io.o -c io.c

cc -DDIAGNOSTICS -g -o utils.o -c utils.c

cc -DDIAGNOSTICS -g -o lib/uni.o -c lib/uni.c

ar -r lib/librng.a lib/uni.o

cc -DDIAGNOSTICS -g main.o schedule.o optimise.o io.o utils.o

    -L./lib -lrng -lm -o scheduler
```

6 Make issues

Make is not portable. When you run the command `make` then, depending on your operating system it could be `gmake`, or GNU Make or Microsoft Windows `nmake`.

The syntax between these versions is not always consistent.

Make relies on timestamps to determine when to rebuild targets. Programs like `tar` that reset timestamps and poor clock synchronisation to file-server may cause problems.

7 Find out more…

The Software Sustainability Institute's build and test examples, (https://github.com/softwaresaved/build_and_test_examples) includes sample Make build scripts for C, C++ and Fortran.